

Chapter 13

Reinforcement learning using human feedback

Reading

1. Training language models to follow instructions with human feedback <https://arxiv.org/abs/2203.02155>
2. Algorithms for Inverse Reinforcement Learning <https://ai.stanford.edu/ang/papers/icml00-irl.pdf>

Our algorithms for state estimation and control relied on the knowledge the system dynamics (LQR, Kalman filter, value iteration), access to a simulator (on-policy, off-policy RL), or access to a reward function (offline RL). These three regimes represent successively weaker assumptions about the availability of data. Let us focus on the third category, where we have some data from the real system and we wish to build a controller that can achieve a slightly better value. In the first problem in Homework 4, you will notice that constructing the correct reward function is the key to achieving good behavior of the learned policy. This is the secret sauce of RL. But it may not be easy to come up with suitable reward models.

For example, imagine the taxi drivers in a busy city. They drive a lot more than other people and they are usually better drivers because they get into fewer accidents. It is difficult for us to guess the reward function that a taxi driver may be using if our goal is to drive a car like them. Even for a problem on picking up objects and placing them in different locations using a manipulator, the reward function is not easy to discover. See the picture below for an example from the talk of Leslie Kaelbling at MIT:

Reward function for pick and place task

hand close to obj	obj grasped	pick ever completed	hand close to goal	place ever completed	reward
0	0	*	*	0	$-\text{distXY}(\text{hand}, \text{obj}) - \text{distZ}(\text{hand}, \text{obj})$
1	0	0	*	0	$-\text{dist}(\text{hand}, \text{obj}) + c1 * \text{closeCmd}$
1	1	0	*	0	$-\text{dist}(\text{hand}, \text{obj}) + c1 * \text{closeCmd} + c2 * \min(\text{obj.z}, \text{targetHt})$
1	1	1	*	0	$-\text{dist}(\text{hand}, \text{obj}) + c1 * \text{closeCmd} + c2 * \text{targetHt} + \text{placeRew}$
1	1	1	0	0	$-\text{dist}(\text{hand}, \text{obj}) + c1 * \text{closeCmd} + c2 * \text{targetHt} + \text{placeRew} + c3 * \text{openCmd}$
*	*	*	*	1	$c2 * \text{targetHt} + \text{placeRew} + c3 * \text{openCmd}$

place ever completed: $|\text{objx} - \text{goalx}| < c4$ and $|\text{objy} - \text{goaly}| < c5$ and $|\text{objz} - \text{goalz}| < c6$
 pick ever completed: $\text{objz} > \text{targetHt} + c7$
 obj grasp: test on current finger sensors
 hand close to obj, hand close to goal: constant x,y,z thresholds
 $\text{placeDist} = |\text{objxy} - \text{goalxy}|$
 $\text{placeRew} = \max(0, c8 * (c9 - \text{placingDist}) + c10 * (\exp(-(\text{placingDist}^{**2}/c11)) + \exp(-(\text{placingDist}^{**2}/c12))))$

It would be very nice to be able to learn reward functions from trajectories of a dynamical system. This is useful because we can then interpret the controller that created these trajectories, and further modify it to suit our purposes using our existing reinforcement learning techniques.

13.1 Inverse Reinforcement Learning

Suppose we have a finite-state MDP with a given transition matrix $T : X \times U \times X \rightarrow \mathbb{R}$ and a reward $r(x)$ for every state. Note that the reward does not depend upon the control. We know that the value function satisfies the Bellman equation

$$v^*(x) = \sum_u q^*(x, u) = \sum_u \left\{ r(x) + \gamma \sum_{x'} T(x' | x, u) v^*(x') \right\}.$$

In particular, if the optimal control at a state x is $u^*(x)$ we have

$$u^*(x) = \operatorname{argmax}_u q^*(x, u) \quad \forall x.$$

Write the vector of rewards corresponding to all states as $\vec{r} \equiv (r(x_1), \dots, r(x_n))$.

Similarly write the vector of the optimal value at all states as $\vec{v}^* \equiv (v^*(x_1), \dots, v^*(x_n))$. We can write the Bellman equation as

$$\begin{aligned} \vec{v}^* &= \vec{r} + \gamma T_{u^*} \vec{v}^*, \\ \vec{v}^* &= (I - \gamma T_{u^*})^{-1} \vec{r} \end{aligned} \tag{13.1}$$

where $T_{u^*} \in \mathbb{R}^{|X| \times |X|}$ be the state transition matrix of the MDP using the optimal control $u^*(x)$ at each state; similarly T_u the state-transition matrix using any other controller $u(x)$. This is the same equation that we have used for policy evaluation. We know that for all $u \in U$

$$\begin{aligned} \sum_{x'} T_{u^*}(x' | x) v^*(x') &\geq \sum_{x'} T_u(x' | x) v^*(x') \\ T_{u^*} \vec{v}^* &\geq T_u \vec{v}^* \\ \Rightarrow T_{u^*} (I - \gamma T_{u^*})^{-1} \vec{r} &\geq T_u (I - \gamma T_{u^*})^{-1} \vec{r}. \end{aligned} \tag{13.2}$$

This inequality is always true because it is simply a rewritten version of the Bellman equation. But it acts a constraint on $\vec{r}$ because it does not depend upon the optimal $\vec{v}^*$.

Suppose we had data from a lot of trajectories of the expert. We could estimate the transition matrix T_{u^*} by positing that the expert is optimal. We could also have an estimate of the transition matrix of T_u . We can now solve

$$\max_{\vec{r}} \|t(\vec{r})\|^2 \quad (13.3)$$

such that $t = (T_{u^*}(I - \gamma T_{u^*})^{-1} - T_u(I - \gamma T_{u^*})^{-1}) \vec{r} \geq 0$.

This is an optimization problem where the variable is the reward function $r(x)$. It is a simple example of inverse reinforcement learning. We can also implement the essential idea for continuous state-control spaces by parameterizing the reward using a neural network and enforcing that the value of any controller u_θ is worse than the optimal value:

$$\forall x : v^*(x) \geq v^\theta(x).$$

Inverse RL is difficult to do in situations when the state-control spaces are infinite because, in principle, we need to enforce the above constraint for every controller u_θ . In practice, you could build a kind of adversarial inverse RL algorithm where the reward is fitted to satisfy this constraint and the controller u_θ is updated to break this constraint. In the next section, we will look at simpler techniques to implement inverse RL.

13.2 Auto-regressive modeling of sequences

Consider the trajectory of a discrete time dynamical system $x \equiv (x_1, x_2, \dots, x_T)$. To learn the model of this system, we may fit the dynamics $x_{t+1} = f(x_t)$ using a model for f , e.g., a neural network with weights w by doing

$$\min_w \frac{1}{N(T-1)} \sum_i \sum_{t=1}^{T-1} \|x_{t+1}^{(i)} - f_w(x_t^{(i)})\|^2.$$

Notice that this objective corresponds to maximum likelihood estimation for the model $x_{t+1} = f(x_t) + \epsilon$ where ϵ is standard Gaussian noise; in other words the probabilistic model of the next state given the past state is a Gaussian. The probability of a trajectory is

$$\begin{aligned} P(x^{(i)}) &= \prod_{t=1}^{T-1} p_w(x_{t+1}^{(i)} | x_t^{(i)}) \\ &= \prod_{t=1}^{T-1} N(x_{t+1}^{(i)}; f_w(x_t^{(i)}), I). \end{aligned}$$

If the system is not Markov with respect to the state x , we may not be able to implement this objective. In such situations, it is useful to model the

likelihood of the trajectory a bit more elaborately, e.g.,

$$\begin{aligned}
 P(x^{(i)}) &= p(x_1^{(i)})p(x_2^{(i)} | x_1^{(i)})p(x_3^{(i)} | x_1^{(i)}, x_2^{(i)}) \dots p(x_T^{(i)} | x_1^{(i)}, \dots, x_{T-1}^{(i)}) \\
 &= \prod_{t=1}^T p(x_t^{(i)} | x_1^{(i)}, \dots, x_{t-1}^{(i)}).
 \end{aligned}
 \tag{13.4}$$

This is called an auto-regressive model of the sequence $x^{(i)}$. We can again parameterize each term using some neural network

$$P(x^{(i)}) = \prod_{t=1}^T p_w(x_t^{(i)} | x_1^{(i)}, \dots, x_{t-1}^{(i)})$$

This is a more general model than the Markov chain model with Gaussian transitions that we talked about above. And therefore it can model many different kinds of sequences—we need not make any assumptions on the state being Markov. Of course, this model is also more difficult to fit because we need a requisitely larger amount of data. Going forward we will write this a bit more succinctly as

$$P(x) = \prod_{t=1}^T p_w(x_t | x_1^t) = \prod_{t=1}^T p_w(x_t | x_{<t}).$$

where x is understood to be a sequence and $x_1^t \equiv (x_1, \dots, x_{t-1})$ is a sub-sequence of length $t - 1$. We will also write things like

$$x_{\geq t} \equiv (x_t, x_{t+1}, \dots, x_T)$$

Self-attention-based networks have become popular to model such sequences. We will not go into the details of how a self-attention-based network works; but roughly speaking, given any past sequence of states x_1^t it fits a model (using cross-entropy loss, so quite similar to supervised learning...) to predict the next state x_t . This amounts to fitting the auto-regressive model above using maximum likelihood estimation.

13.3 Human feedback

Suppose we collected a dataset $D = \{x^{(i)}\}_{i=1}^N$ of a lot of sequences; for sentences this could be as large as most of the Internet, for trajectories of dynamical systems this could be a dataset of as many dynamical systems as you can lay your hands on. Let us focus on sentence because they are popular. After fitting a self-attention-based model to this data, we could prompt this model using a few initial words and run the model forward to complete the sentence:

I am a boy. My name is Pratik.

1 This is done by sampling

$$\begin{aligned}\hat{x}_t &\sim P(x_t \mid x_{<t}) \\ \hat{x}_{t+1} &\sim P(x_{t+1} \mid x_{<t}, \hat{x}_t) \\ &\vdots\end{aligned}$$

2 just like we would draw samples from a Markov chain. If we ingest very
3 large datasets, we do not have good control on the quality of sentences
4 that our model generates. For example, it could complete the prompt
5 with something that is toxic (since most of the Internet contains such
6 sentiments...). We would like to prevent our model from completing
7 sentences in such a toxic manner.

A self-attention-based model of sentences (language model) is a dynamical system. If we would construct some reward function then could do reinforcement learning, or something like it, to modify the transition probabilities of the Markov chain to ensure that it completes the sentences in a non-toxic fashion. Effectively, we would like to sample sentences from a slightly different model

$$x \sim P(x)r(x) \tag{13.5}$$

than $P(x)$ which was built from the dataset D . Our goal will be to build such a “reward function” $r(x)$.

Bibliography